

Sintaxis de C-

Una gramática BNF para C- es como se describe a continuación:

1. $program \rightarrow declaration\text{-}list$
2. $declaration\text{-}list \rightarrow declaration\text{-}list\ declaration \mid declaration$
3. $declaration \rightarrow var\text{-}declaration \mid fun\text{-}declaration$
4. $var\text{-}declaration \rightarrow type\text{-}specifier\ ID \ ; \mid type\text{-}specifier\ ID \ [\ NUM \] \ ;$
5. $type\text{-}specifier \rightarrow int \mid void$
6. $fun\text{-}declaration \rightarrow type\text{-}specifier\ ID \ (\ params \) \ compound\text{-}stmt$
7. $params \rightarrow param\text{-}list \mid void$
8. $param\text{-}list \rightarrow param\text{-}list \ , \ param \mid param$
9. $param \rightarrow type\text{-}specifier\ ID \mid type\text{-}specifier\ ID \ [\]$
10. $compound\text{-}stmt \rightarrow \{ \ local\text{-}declarations \ statement\text{-}list \}$
11. $local\text{-}declarations \rightarrow local\text{-}declarations \ var\text{-}declaration \mid empty$
12. $statement\text{-}list \rightarrow statement\text{-}list \ statement \mid empty$
13. $statement \rightarrow expression\text{-}stmt \mid compound\text{-}stmt \mid selection\text{-}stmt$
 $\mid iteration\text{-}stmt \mid return\text{-}stmt$
14. $expression\text{-}stmt \rightarrow expression \ ; \ ;$
15. $selection\text{-}stmt \rightarrow if \ (\ expression \) \ statement$
 $\mid if \ (\ expression \) \ statement \ else \ statement$
16. $iteration\text{-}stmt \rightarrow while \ (\ expression \) \ statement$
17. $return\text{-}stmt \rightarrow return \ ; \mid return \ expression \ ;$
18. $expression \rightarrow var = expression \mid simple\text{-}expression$
19. $var \rightarrow ID \mid ID \ [\ expression \]$
20. $simple\text{-}expression \rightarrow additive\text{-}expression \ relop \ additive\text{-}expression$
 $\mid additive\text{-}expression$
21. $relop \rightarrow < = \mid < \mid > \mid > = \mid == \mid !=$
22. $additive\text{-}expression \rightarrow additive\text{-}expression \ addop \ term \mid term$
23. $addop \rightarrow + \mid -$
24. $term \rightarrow term \ mulop \ factor \mid factor$
25. $mulop \rightarrow * \mid /$
26. $factor \rightarrow (\ expression \) \mid var \mid call \mid NUM$
27. $call \rightarrow ID \ (\ args \)$
28. $args \rightarrow arg\text{-}list \mid empty$
29. $arg\text{-}list \rightarrow arg\text{-}list \ , \ expression \mid expression$

Gramática escrita para un EBNF basado en la proporcionada:

1. $program \rightarrow declaration\text{-}list$
2. $declaration\text{-}list \rightarrow declaration \{ declaration \}$
3. $declaration \rightarrow type\text{-}specifier\ ID \ (\ var\text{-}declaration \mid fun\text{-}declaration \)$

4. var-declaration -> ; | [NUM] ;
5. type-specifier -> int | void
6. fun-declaration -> (params) compound-stmt
7. params -> param-list | void
8. param-list -> param { , param }
9. param -> type-specifier ID [[]]
10. compound-stmt -> { local-declarations statement-list }
11. local-declarations -> { var-declaration }
12. statement-list -> { statement }
13. statement -> expression-stmt | compound-stmt | selection-stmt | iteration-stmt |
return-stmt
14. expression-stmt -> [expression] ;
15. selection-stmt -> if (expression) statement [else statement]
16. iteration-stmt -> while (expression) statement
17. return-stmt -> return [expression] ;
18. expression -> var = expression | simple-expression
19. var -> ID [[expression]]
20. simple-expression -> additive-expression [relop additive-expression]
21. relop -> <= | < | > | >= | == | !=
22. additive-expression -> term { addop term }
23. addop -> + | -
24. term -> factor { mulop factor }
25. mulop -> * | /
26. factor -> (expression) | var | call | NUM
27. call -> ID (args)
28. args -> arg-list | ϵ (vacío)
29. arg-list -> expression { , expression }